词法分析实验 AI 使用记录

AI工具：Cursor。我负责 `lexer.l` 及共同接口联调。

成员A：胡家华2410935

成员B：马禹翔2410933

分工原则

- **AI**：总体架构、需要改哪些文件、Token 清单、Flex 规则应分成哪几类、每个函数的职责和调用关系。
- **人工**：函数体内部实现，包括进制转换、行列号计算、Token 封装时的分支与异常处理，以及规则书写顺序的最终核对。

下文各节后附与大模型的对话截图，对应本次实验中实际使用的提问与回答。

1. 需求

输入一份 SysY 源文件，按源码顺序输出单词。空白和注释不输出，但要维护后续 Token 的行号、列号。每个 Token 输出类别、词素、属性、行号、列号。

提问：本次实验的输入、输出和约束是什么？单词的行号列号如何确定？



约束

- 用 Flex 写匹配规则；空白、`//`、`/* */` 要能扫过，但不能当 Token 输出。
- 整型要能识别十进制、八进制、十六进制，词素保留原文，属性是十进制值（`012` → 属性 `10`）。
- `-2147483648` 拆成 `MINUS` 和 `LL_CONST 2147483648`，负号单独一个单词。
- 浮点常量要识别，属性按 `float` 输出。
- 多行注释到第一次 `*/` 结束；注释后第一个单词的位置必须仍然正确。
- 先最长匹配，长度相同时用规则书写顺序（所以关键字要写在标识符前面）。
- 不写完整实验报告；小雅交源程序或 Git 链接；线下演示并回答提问。

行号、列号怎么定

记的是**词素第一个字符**的位置，不是匹配结束位置。

- **行号从 1 开始。** 每读到换行，行号加 1，列回到行首。
- **列号从 0 开始。** 这是指导书示例的约定。框架里 Bison 的 `location` 列往往从 1 计，转成输出时要减 1。
- 空格、换行、Tab、注释虽然不输出，也必须推进位置，否则后面单词的行列会错。
- Tab 按框架宽度 4 对齐，不能按「Tab 只占 1 列」来算。
- 多行注释里的换行同样要增加行号，这样注释结束后的第一个 Token 才会落在正确的行。

例如指导书里的 `int INT_MIN = -2147483648;`：`INT` 在第 1 行第 0 列，`IDENT INT_MIN` 在第 1 行第 4 列（`int` 后面有一个空格）。

2. AI 给出的架构与文件划分

AI 根据课程框架和接口约定，确定仍使用 `NKU-Compiler2025`，不另写独立 Lexer。文件职责如下。

文件	分工	在架构中的作用
`frontend/parser/lexer.l`	成员 A	正规定义、匹配规则、位置维护、调用转换函数
`frontend/parser/yacc.y`	共同 / 成员 B	声明 Token 名称和属性类型
`interfaces/frontend/token.h`	成员 B	`FE::Token` 的类别、词素、属性、位置
`frontend/parser/parser.cpp`	共同联调	把 Bison Token 转成五列输出所需的 `FE::Token`
`testcase/lexer/`	共同验证	`simple`、`witharray`、`withfloat`

处理流水线由 AI 归纳为：

```
.sy 源文件
    -> Scanner::nextToken()          // lexer.l 规则
    -> convertTo* / make*            // 词素变数值，再变成 Bison Token
    -> Parser::parseTokens_impl()    // 填 FE::Token
    -> main.cpp -lexer               // 打印五列
```

提问：数据从 .sy 到五列输出需要经过哪些模块？项目目录如何划分、哪些文件需要更改？

请给出词法分析的总体架构：
数据从 .sy 到五列输出需要经过哪些模块？
可以给我设计一下项目的目录结构吗？有哪些文件需要更改？

Thought 1s >

不要另起一套目录。继续用课程框架 `NKU-Compiler2025`，数据从 `.sy` 走到五列输出，只经过前端这一条线。

总体架构

```
test.sy
    |
    ├─ main.cpp 识别 -lexer，打开输入/输出流
    |
    ├─ Parser 构造 Scanner，把流交给 Flex
    |
    ├─ lexer.l / Scanner::nextToken()
    |     按规则切出 Bison Token
    |     空白、注释只更新 location，不返回 Token
    |     常量调用 convertTo*，再变成带属性的 Token
    |
    ├─ parser.cpp / parseTokens_impl()
    |     取出类别名、词素、location
    |     列号减 1，变成从 0 计
    |     按种类填 ival / lval / fval / sval
    |     得到 FE::Token 数组
    |
    └─ main.cpp 按固定宽度打印五列
          Token  Lexeme  Property  Line  Column
```

词法阶段**不会**走到语法树、中端 IR 或后端汇编。`yacc.y` 在这一步只提供 Token 名称和属性类型；真正规约文法是后面语法分析才做。

需要改的文件

文件	改什么	谁
`frontend/parser/lexer.l`	正规定义、匹配顺序、注释状态、调用转换函数、非法字符	成员 A（主文件）
`frontend/parser/yacc.y`	补上 `INT`、`FLOAT`、`LL_CONST`、`FLOAT_CONST`、运算符等 `%token`，**不改文法**	共同，否则 A 的规则编不过
`frontend/parser/parser.cpp`	`parseTokens_impl` 里为超范围整数和浮点填属性	共同联调
`interfaces/frontend/token.h`	确认已有 `token_name/lexeme/line/column` 和 `T_INT/T_LL/T_FLOAT/T_STRING`	成员 B，框架已基本具备

不要改： `main.cpp` 的打印格式（列宽已与 `.lexer` 对齐）、`middleend/`、`backend/`、官方三个 `.lexer` 期望文件。

模块怎么串起来

1. `lexer.l` 认出一个词素，带上 `location`（Bison 列从 1 计）。
2. 若是常量，先变成数值，再 `make_INT_CONST` / `make_LL_CONST` / `make_FLOAT_CONST`。
3. `parser.cpp` 把 `token.name()` 写成 Token 列，`YYText()` 写成 Lexeme 列，`begin.line` 写成 Line，`begin.column - 1` 写成 Column。
4. `main.cpp` 按 `type` 决定 Property 打印数字还是标识符字符串，还是留空。

3. AI 确定的函数与职责

AI 列出需要出现的函数，并约定谁实现函数体。声明、参数和返回值由架构讨论确定；函数体由人工编写或沿用框架实现。

函数	所在位置	AI 确定的职责	函数体
YY_USER_ACTION / handleTab()	lexer.l	每个词素先 step 再按长度推进列；Tab 宽度 4	框架已有，人工核对后调用
convertToInt(str, end, isLongLong)	lexer.l	识别 10/8/16 进制，属性为十进制值；超出 int 时置 isLongLong	框架已有，人工阅读后调用
convertToFloatDec(str)	lexer.l	十进制浮点词素转为 float	框架已有，人工阅读后调用
convertToFloatHex(str)	lexer.l	十六进制浮点词素转为 float，指数按 2 的幂	框架已有，人工阅读后调用
makeIntConst(text, loc, parser)	lexer.l	调用 convertToInt，在 INT_CONST 与 LL_CONST 之间选择	人工编写函数体
makeFloatDec(text, loc, parser)	lexer.l	调用 convertToFloatDec，失败则 ERR_TOKEN	人工编写函数体

函数	所在位置	AI 确定的职责	函数体
`makeFloatHex(text, loc, parser)`	`lexer.l`	调用 `convertToFloatHex`，失败则 `ERR_TOKEN`	人工编写函数体
`Parser::parseTokens_impl()`	`parser.cpp`	按 Token 种类填写 `ival` / `lval` / `fval` / `sval`	在原有 switch 中由人工补 `LL_CONST`、`FLOAT_CONST` 两支
`Parser::reportError()`	`parser.cpp`	把词法错误位置交给 Bison	框架已有

Flex 规则不做成函数，但由 AI 划分类别，规则动作由人工填写：

1. 空白、换行、Tab
2. 单行注释、`COMMENT` 独占状态
3. 关键字（写在标识符之前）
4. 多字符运算符，再写单字符运算符
5. 浮点规则，再写整型规则
6. 标识符、字符串
7. 非法字符与未闭合注释

提问：lexer.l 和 parser.cpp 里需要出现哪些函数？请命名并介绍功能。



需要你写函数体的封装

规则动作里不要堆转换和 try/catch，收到词素后只调这三个：

名字	作用
`makeIntConst(text, loc, parser)`	调用 `convertToInt`。`isLongLong` 为假则生成 `INT_CONST`，为真则生成 `LL_CONST`。转换失败报告错误并生成 `ERR_TOKEN`。
`makeFloatDec(text, loc, parser)`	调用 `convertToFloatDec`，生成 `FLOAT_CONST`；失败则 `ERR_TOKEN`。
`makeFloatHex(text, loc, parser)`	调用 `convertToFloatHex`，生成 `FLOAT_CONST`；失败则 `ERR_TOKEN`。

另外还有宏，不算新函数，但和位置有关：

- `YY_USER_ACTION`：每条规则开头先 `loc.step()`，再按 `yyleng` 增加列。
- `RETT(名字, loc)`：没有额外属性的 Token（关键字、运算符、括号）直接 `return make_XXX(loc)`。

Flex 规则本身不是函数。建议按这个顺序写，先比匹配长度，长度相同再看谁写在前面：

1. 换行、空格、Tab（不输出，只改位置）
2. `//` 单行注释；`/*` 进入独占状态 `COMMENT`
3. 关键字（必须在标识符前面）
4. 多字符运算符 `==` `!=` `<=` `>=` `&&` `||`，再写单字符
5. 浮点（十进制、十六进制），再写整数（十六进制、八进制、十进制）
6. 标识符、字符串
7. 非法字符；注释未闭合在 `COMMENT` 的文件结束处处理

parser.cpp

名字	作用
`Parser::parseTokens_impl()`	词法阶段的核心。循环调用 `nextToken()`，直到 `END`。每个 Token 填五列：`name()` 是类别，`YYText()` 是词素，`begin.line` 是行号，`begin.column - 1` 是列号。再按种类把属性写入 `ival` / `lval` / `fval` / `sval`。
`Parser::reportError(loc, message)`	框架已有。词法出错（非法字符、转换失败、注释未闭合）时由 `lexer.l` 调用，把位置和信息交给 Bison。
`Parser::parseAST_impl()`	语法分析用，本实验不要改。

`parseTokens_impl` 里属性怎么填：

- `INT_CONST` → 整型数值
- `LL_CONST` → `long long`（例如 `2147483648`）
- `FLOAT_CONST` → 浮点数值
- `IDENT` / `ERR_TOKEN` / 字符串 → 字符串属性
- 关键字、运算符、括号 → 无属性，Property 留空

4. 人工实现时依据的约定

这些判断由指导书和接口约定确定，写进函数体或规则动作时不交给 AI 自行发挥：

- `-2147483648` 拆成 `MINUS` 和 `LL_CONST 2147483648`
- `012` 词素保留原文，属性为 10；`0xA` 同理
- `0x.Ap-3` 必须走浮点规则，不能当成非法字符或整数
- 注释内容不进入 Token 流
- 输出列号使用 `begin.column - 1`

5. AI 辅助调试

AI 帮助设计验证步骤，不代替对照期望文件：

```
make
bin/compiler -lexer -o simple.out testcase/lexer/simple.sy
diff -u testcase/lexer/simple.lexer simple.out
```

`simple`、`witharray`、`withfloat` 由人工确认五列一致。

提问：正规定义、匹配规则、位置维护、调用转换函数是否已经完全实现？如何测试？

好的，我在本次实验负责正规定义、匹配规则、位置维护、调用转换函数，这些功能是否已经完全实现？如何进行测试？

Worked for 1m 13s

Thought 1s

我对照你负责的四块看一下 `lexer.l` 现在缺不缺，再告诉你怎么测。

Explored lexer.l, 1 search

你负责的四块在 `lexer.l` 里已经齐了，可以进入测试，不必再扩架构。

职责	现状
正规定义	已有 `ident`、`dec_int` / `oct_int` / `hex_int`、`dec_float` / `hex_float`
匹配规则	关键字在标识符前；浮点在整数前；`==` 等长运算符在单字符前；非法字符走 `ERR_TOKEN`
位置维护	`YY_USER_ACTION` 推进列；`\n` 换行；Tab 调 `handleTab`；`COMMENT` 里同样更新行列
调用转换	整数走 `makeIntConst` → `convertToInt`；十进制/十六进制浮点分别走 `makeFloatDec` / `makeFloatHex`

如何测试

在 WSL 的 Ubuntu 里进入框架目录后编译：

```
cd "/mnt/c/Users/hujiahua/大三上/编译系统原理/编译原理大作业/词法分析器/NKU-Compiler2025"
make
```

对三个官方样例做五列对比，**输出应与期望文件完全一致**：

```
bin/compiler -lexer -o simple.out testcase/lexer/simple.sy
diff -u testcase/lexer/simple.lexer simple.out

bin/compiler -lexer -o witharray.out testcase/lexer/witharray.sy
diff -u testcase/lexer/witharray.lexer witharray.out

bin/compiler -lexer -o withfloat.out testcase/lexer/withfloat.sy
diff -u testcase/lexer/withfloat.lexer withfloat.out
```

没有 `diff` 输出就通过。这三项分别覆盖：

- `simple`：关键字、标识符、`<=`、`-2147483648` → `MINUS` + `LL_CONST`
- `witharray`：`[ ]`、关系运算、单行/多行注释后的行列号
- `withfloat`：`3.14`、`.5`、`1e10`、`0x1.8p0`、`0x.Ap-3` 一类浮点

补充样例（注释组合、Tab、非法字符、未闭合 `/*`）：

```
bin/compiler -lexer -o comment.out testcase/lexer/comment_cases.sy
```

看输出里注释是否消失、`g/**//h` 是否变成 `g` `/` `h`、`@` 是否为 `ERR_TOKEN`。这一项没有官方 `.lexer`，人工看即可。